

Data drift and monitoring

A deployed model is frozen; the data arriving at it is not. Compare the two, feature by feature, and act early.

THE QUESTION

Why does a model that scored well at launch get worse when nobody touched the code, and how do you notice in time?

THE INTUITION

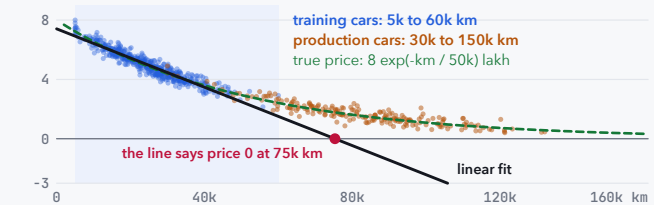
A model is a summary of old data. It holds while new inputs look like the old ones, the rule from input to label stays, and the label mix stays put. Monitoring flags gaps between new and training data too large to be chance.

KEY TAKEAWAY

Ask: did X change, did $X \rightarrow Y$ change, did the mix of Y change? Plot first, then KS or PSI for numbers and chi-squared for categories; check labelled accuracy when labels arrive.

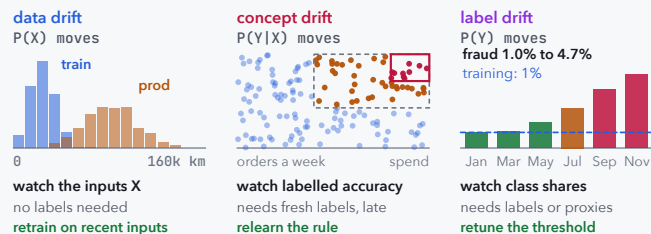
train → deploy → monitor inputs → KS · PSI · χ^2 → alert → diagnose → retrain

1 New inputs break an unchanged model



A line fit on cars with 5,000 to 60,000 km meets cars with 30,000 to 150,000 km. The pricing rule has not changed, but the line extrapolates; an R^2 below 0 is worse than predicting the mean price.

2 X , the rule, or the label mix can move



Data drift is also called covariate shift. After a free-delivery plan, 4 orders a week no longer means premium, and 39 of these 150 customers change label. Label drift keeps the classes but changes their shares.

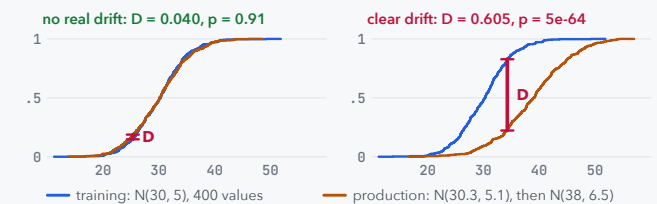
3 Rule out a broken pipeline, then plot

symptom	shape (sketch)	more likely
a feature's mean creeps up for weeks		drift: test it
new kinds of users arrive slowly		drift: test it
mileage is 0 for every car since Tuesday		bug: fix the pipeline
a price column is 100x overnight		bug: fix the pipeline

Then overlay each feature from both sides. Often only some features move: more old cars shift mileage and car age, while the brand and fuel mix may hold.

```
plt.hist(train["km"], 30, alpha=0.5)
plt.hist(prod["km"], 30, alpha=0.5)
```

4 KS: the largest gap between two CDFs

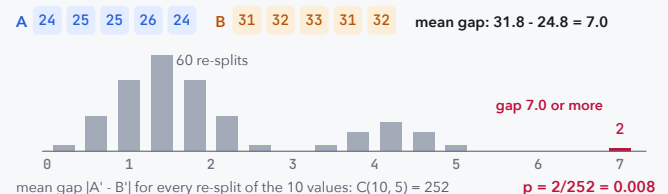


A CDF gives the share of values $\leq x$. D runs from 0 (same) to 1 (no overlap); the car mileages give 0.945. KS needs numeric values; run it per feature.

```
from scipy.stats import ks_2samp
stat, p = ks_2samp(train_km, prod_km)
```

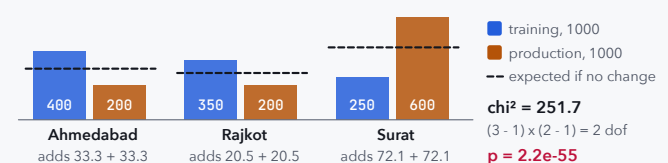
SciPy's p is exact here (0.907); the series $2 \sum_{k \geq 1} (-1)^{k-1} e^{-2k^2 \lambda^2}$, $\lambda = D \sqrt{nm/(n+m)}$, gives 0.906.

5 A p-value: how often chance does as well



If nothing changed, any split of the pooled values is as likely as the real one; KS does the same with D . $p < 0.05$ is a common warning line. It means the gap would be rare without drift; it is not the chance that drift is real.

6 Chi-squared compares category counts



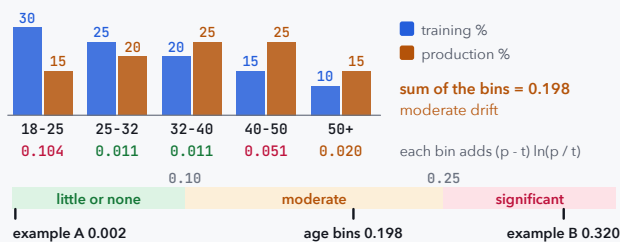
Cities have no order, so no CDF. Expected = row total × column total / grand total; each cell adds $(O - E)^2 / E$. Labels are categories too.

```
from scipy.stats import chi2_contingency
chi2, p, dof, expected = \
    chi2_contingency(table)
```

From one test to a monitoring loop

PSI gives one score per feature, Evidently runs every check at once, and a weekly job with thresholds turns checks into alerts.

7 PSI adds up the shift in each bin



PSI = $\sum_b (p_b - t_b) \ln(p_b / t_b)$, with training shares t_b and production shares p_b . Under 0.10: little or none; 0.10 to 0.25: moderate; over 0.25: significant.

8 Bins and empty bins change the number

bin	train	prod	adds to PSI (floor 1e-6)	floor .001
15-25	15.0%	2.4%	0.232	0.232
25-32	51.7%	15.2%	0.447	0.447
32-40	30.7%	44.0%	0.048	0.048
40-50	2.6%	33.2%	0.779	0.779
50-65	0.0%	5.2%	0.565	0.202
PSI			2.070	1.707

training deciles as bins (compute_psi below): PSI = 1.495

```
def compute_psi(ref, cur, bins=10):
    e = np.quantile(ref,
                    np.linspace(0, 1, bins + 1))
    e[-1] += 1e-6
    r = np.histogram(ref, e)[0] / len(ref)
    c = np.histogram(cur, e)[0] / len(cur)
    r = np.clip(r, 0.001, None)
    c = np.clip(c, 0.001, None)
    return np.sum((c - r) * np.log(c / r))
```

An empty training bin makes $\ln(p/0)$ infinite, so the floor sets its share. Training deciles leave no bin empty.

9 Evidently checks every column at once

the same checks by hand, on the Evidently companion's data (300 rows each)

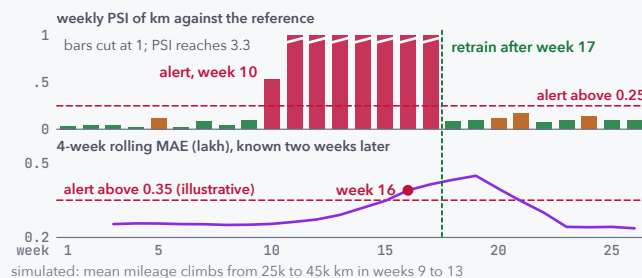
column	kind	test	statistic	p-value	drift
age	numeric	KS	D = 0.607	2e-48	yes
income	numeric	KS	D = 0.287	4e-11	yes
city	category	chi-squared	101.2	1e-22	yes

3 of 3 columns drifted at $p < 0.05$, so the dataset counts as drifted

```
from evidently.report import Report
from evidently.metric_preset import \
    DataDriftPreset
report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=df_train,
           current_data=df_prod)
report.save_html("drift_report.html")
```

One HTML page, a suitable test per column. This is the older API; current releases import Report from evidently itself, so check your version's docs.

10 Monitor on a schedule, alert on a threshold



A weekly job (cron or GitHub Actions) checks that week's data against the reference and saves the report. PSI fires at week 10; the price error waits for sale prices and crosses its line at week 16.

11 After an alert: diagnose, then act

cause found	what to do
a pipeline bug	fix the pipeline, not the model
a seasonal pattern	expected; maybe a model per season
a new user segment	collect labels for it, retrain
the world changed	a major retrain on new data

Then match the fix to the drift type (panel 2). A 0.80 review cutoff set for 1% fraud approves many risky 0.6 to 0.8 scores once fraud is 5%. Retrain with pinned packages and seeds, and compare v1 with v2 (see the reproducible runs sheet).

12 Pause and think

Accuracy rose under drift? A loan model scored 81.5% clean, 74.5% with one drifted feature and 83.5% with three. If 95% deserve a reject, "reject all" scores 95%.

$p = 0.01$: 99% sure it drifted? No. A gap this big would appear about 1 time in 100 if nothing had changed.

Histograms steady, complaints rising? Suspect concept drift: same X , new Y . Only labelled checks catch it.

Every feature flagged on huge data? p shrinks as rows grow, so tiny shifts get tiny p . Read the size too: D or PSI.

No labels for weeks? Watch X now; until labels arrive, track predicted class shares or audit a sample by hand.

Drift flagged, accuracy fine? Not all drift is bad: more users can widen the data. Diagnose before you retrain.

plot first · KS or PSI for numbers · chi-squared for categories · labels for concept drift · alert, diagnose, then retrain

Source: lecture "Data drift and model monitoring" with its drift-response section, its companion notebooks (p-value, KS, chi-squared, PSI, Evidently) and the drift notebook, nipunbatra.github.io/stt-ai-teaching. Numbers are recomputed in numpy from the lecture's code and seeds (car fit, KS with exact and asymptotic p , the 252 re-splits, chi-squared, PSI, the Evidently companion's data); the loan accuracies are quoted; the fraud months and monitoring weeks are simulated. Next: the web apps and APIs sheet.